

WIA1006/WID3006

MACHINE LEARNING

Semester 2, Session 2025/2026 | Universiti Malaya

Home of the Bright, Land of the Brave
Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani

www.um.edu.my



UNIVERSITI
MALAYA

CLUSTERING

Home of the Bright, Land of the Brave
Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani



www.um.edu.my



[universityofmalaya](https://www.facebook.com/universityofmalaya)



[unimalaya](https://www.instagram.com/unimalaya)



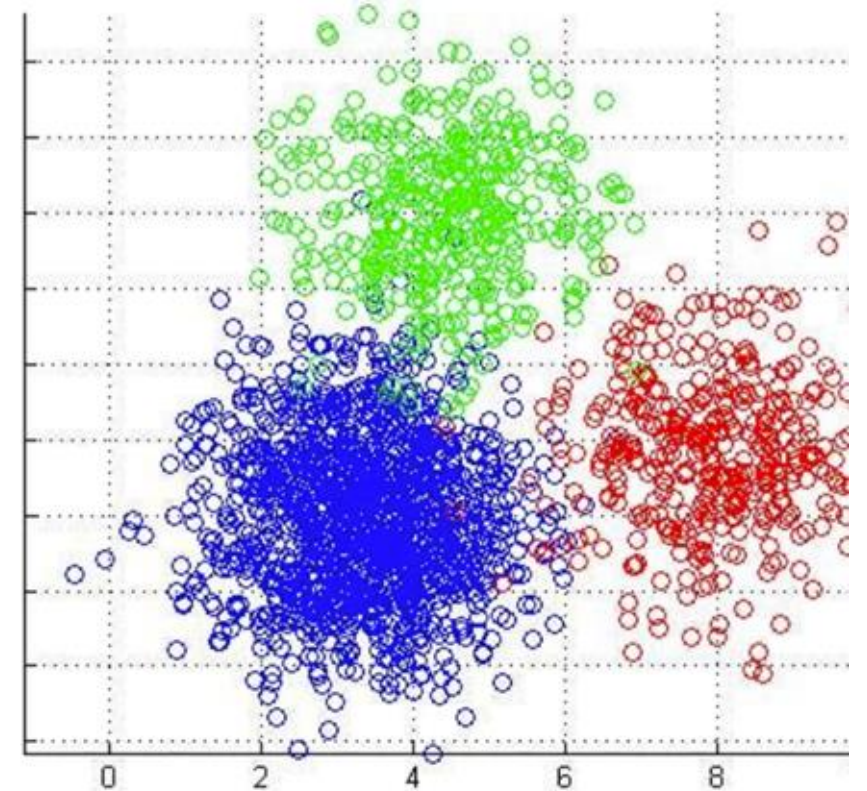
[uniofmalaya](https://www.youtube.com/uniofmalaya)



UNIVERSITI
MALAYA

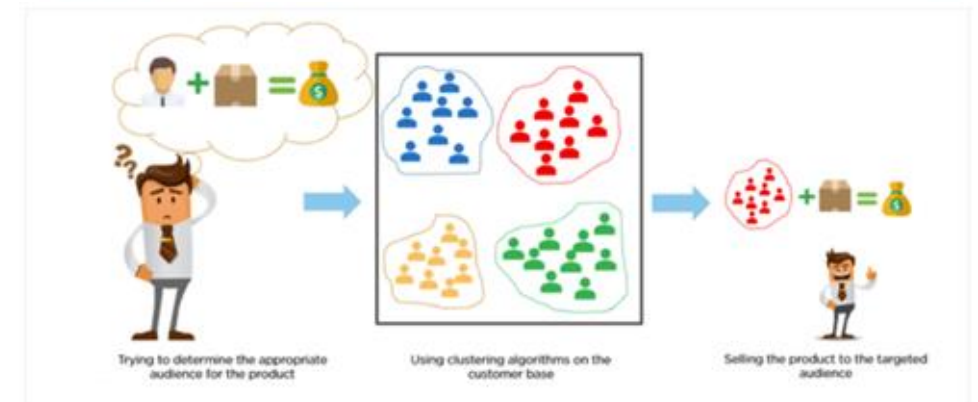
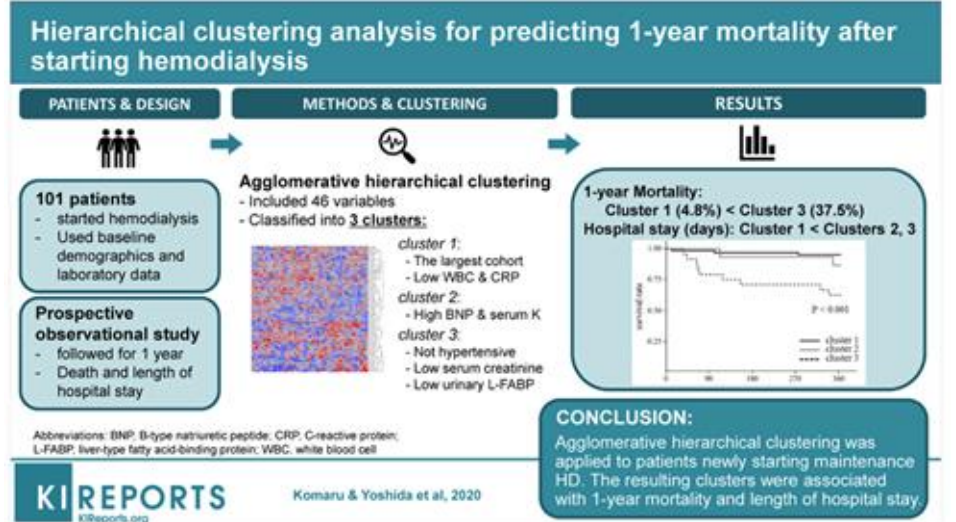
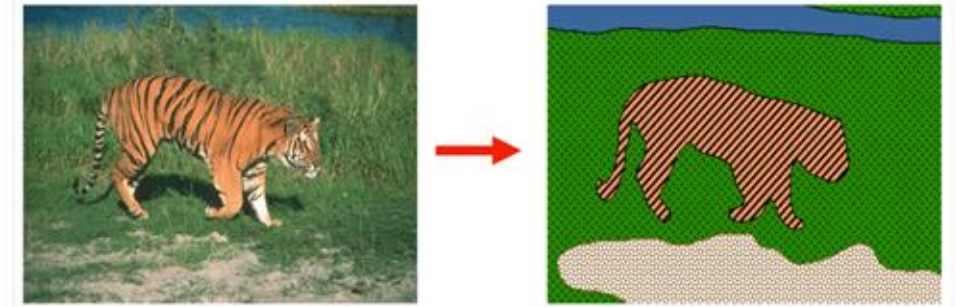
INTRODUCTION

- Clustering is a machine learning technique that involves grouping similar objects or data points together into clusters, based on certain features or characteristics.
- Important in various industries, such as marketing, healthcare, finance, and more.
- The **main goals** of clustering are to identify natural groupings in data, minimize the distance or dissimilarity between data points within the same cluster, and maximize the distance or dissimilarity between data points in different clusters.



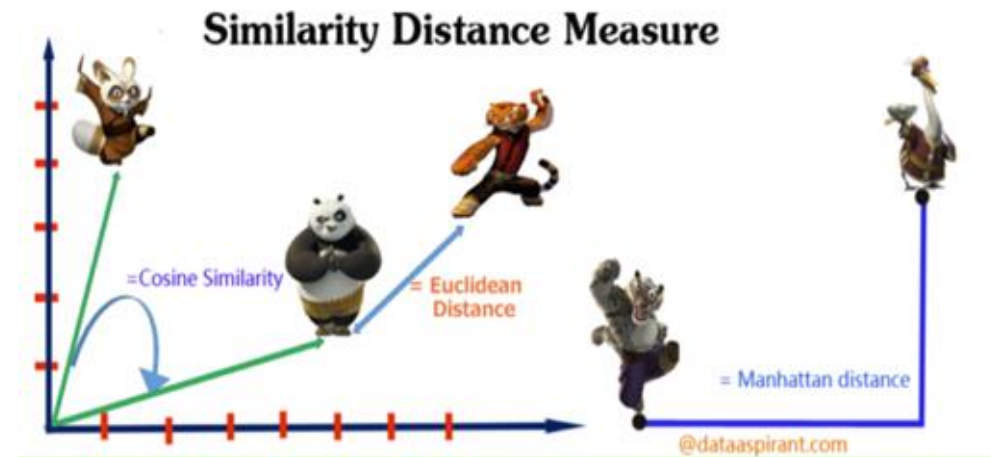
APPLICATIONS OF CLUSTERING

- Marketing
- Healthcare
- Finance
- Image and speech recognition
- Anomaly detection



SIMILARITY AND DISSIMILARITY MEASURES FOR CLUSTERING

- **Euclidean Distance:** This is a common measure of distance between two points in a multidimensional space. It is calculated as the square root of the sum of the squared differences between corresponding coordinates of the two points.
- **Manhattan Distance:** This is another measure of distance between two points, based on the absolute differences between corresponding coordinates of the two points. It is often used in image recognition or pattern matching.
- **Cosine Similarity:** This is a measure of similarity between two vectors, based on the cosine of the angle between them. It is often used in natural language processing or recommendation systems.



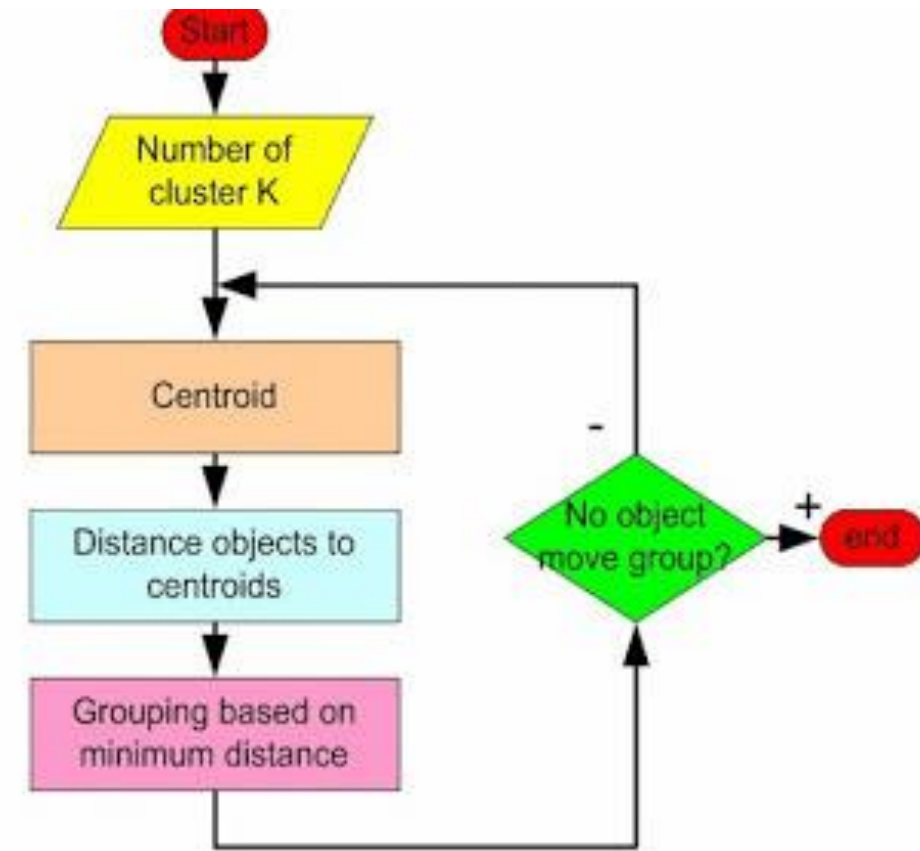
CLUSTERING ALGO: K-MEANS

- K-Means is a widely used algorithm for clustering that aims to partition a dataset into K distinct clusters, where K is a pre-specified number.
- Effective for well-separated clusters and can handle different types of data, such as numerical or categorical variables.
- Requires the pre-specification of the number of clusters, which may not be known in advance or may be difficult to determine.

K-MEANS ALGORITHM

- Steps of K-Means Algorithm:

1. Initialization: Select K random data points from the dataset as the initial centroids for the K clusters.
2. Assignment: Assign each data point to the cluster whose centroid is closest to it, based on a distance measure such as Euclidean distance or Manhattan distance.
3. Update: Recalculate the centroid of each cluster as the mean of all the data points assigned to it in the previous step.
4. Convergence: Repeat steps 2 and 3 until the centroids converge, that is, until the change in the centroids is below a certain threshold or the maximum number of iterations is reached.



K-MEANS ALGORITHM - INITIALIZATION

- The initialization step is the first step in the K-Means clustering algorithm, and it involves selecting the initial centroids for the K clusters.
- One common method for initialization is random selection, where K data points are randomly selected from the dataset to serve as the initial centroids.
- Example:
 - As a starting point, you tell your model how many clusters it should make. First the model picks up K, (let $K = 3$) datapoints from the dataset. These datapoints are called cluster centroids.
 - Now there are different ways you to initialize the centroids, you can either choose them at random — or sort the dataset, split it into K portions and pick one datapoint from each portion as a centroid.

K-MEANS ALGORITHM – ASSIGNMENT STEP

- Involves assigning each data point to the nearest centroid based on its distance.
- The distance metric used to measure the distance between data points and centroids can affect the resulting clusters
- Most common distance metric used in K-Means clustering is the Euclidean distance

$$D(x_i, x_{i'}) = \|x_i - x_{i'}\| = \sqrt{\sum_{j=1}^p (x_{ij} - x_{i'j})^2}$$

- Clusters defined by Euclidean distance is invariant to translations and rotations in feature space, but not invariant to scaling of features.

L2 Distance a.k.a Euclidean distance

$$\text{dist} = (x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2$$

	centroids	datapoint	
c1	2 3 1	4 2 0	Assign a cluster to data point
c2	8 7 2	$(4-2)^2 + (2-3)^2 + (0-1)^2 = 6$	datapoint belongs to c1 cuz 6 is minimum
		$(4-8)^2 + (2-7)^2 + (0-2)^2 = 45$	
c3	5 6 0	$(4-5)^2 + (2-6)^2 + (0-0)^2 = 17$	

K-MEANS ALGORITHM – UPDATE STEP

- The update step is the third step in the K-Means clustering algorithm, and it involves updating the centroids of each cluster based on the mean of the data points assigned to that cluster.
- The most common method used for updating centroids in K-Means clustering is the simple mean, which calculates the mean of all the data points assigned to each cluster.
- Because the initial centroids were chosen arbitrarily, your model updates them with new cluster values.
- If some other algo, like K-Mode, or K-Median was used, instead of taking the average value, mode and median would be taken respectively.

Updating Cluster Centroids

old centroid of C#1:

2	3	1
---	---	---

f1	f2	f3
4	2	0
3	3	1
5	1	3
4	0	2

} datapoints in C#1

New centroid = Avg of data points feature wise

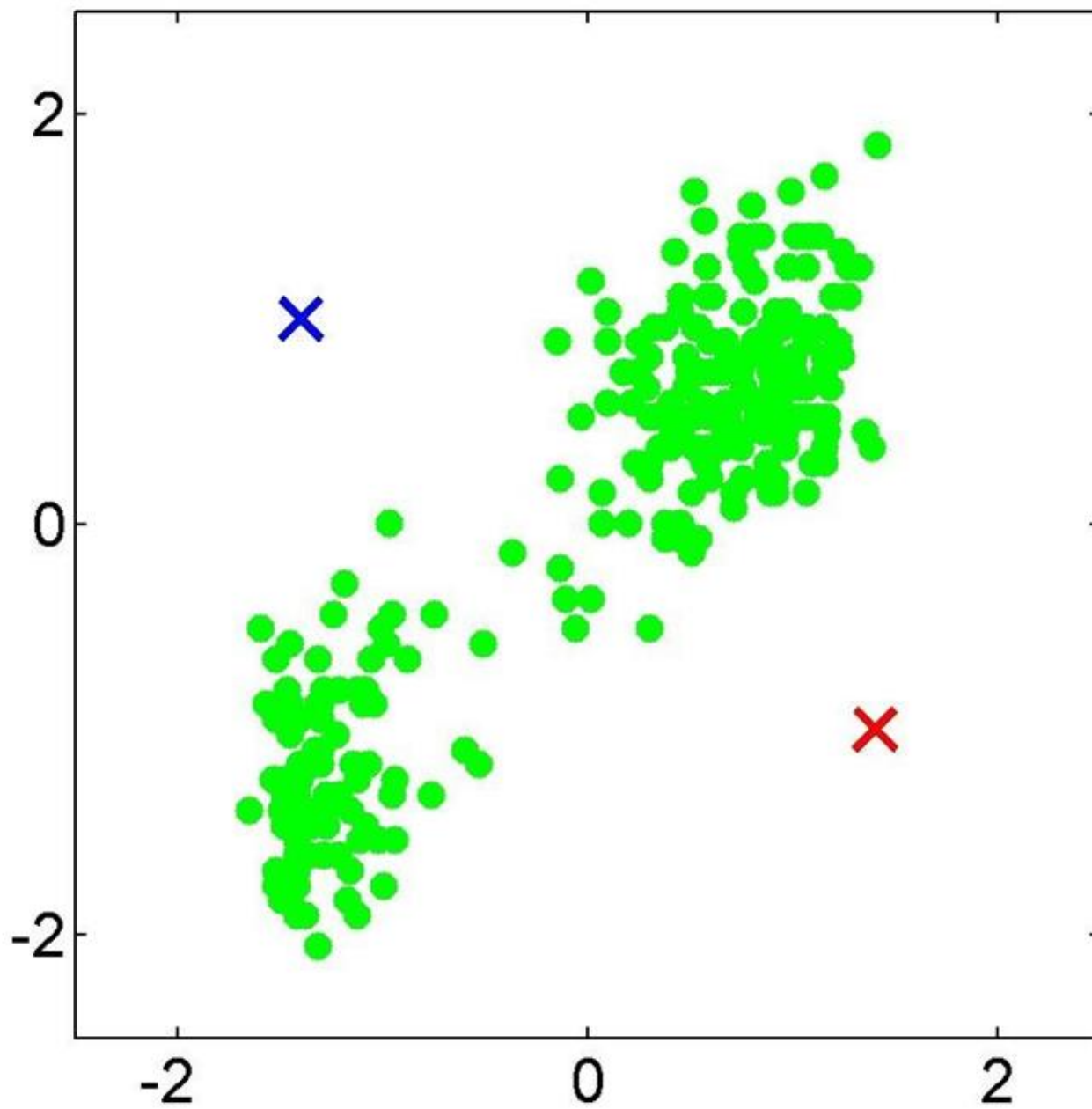
$$\frac{4+3+5+4}{4}=4, \frac{2+3+1+0}{4}=1.5, \frac{0+1+3+2}{4}=1.5$$

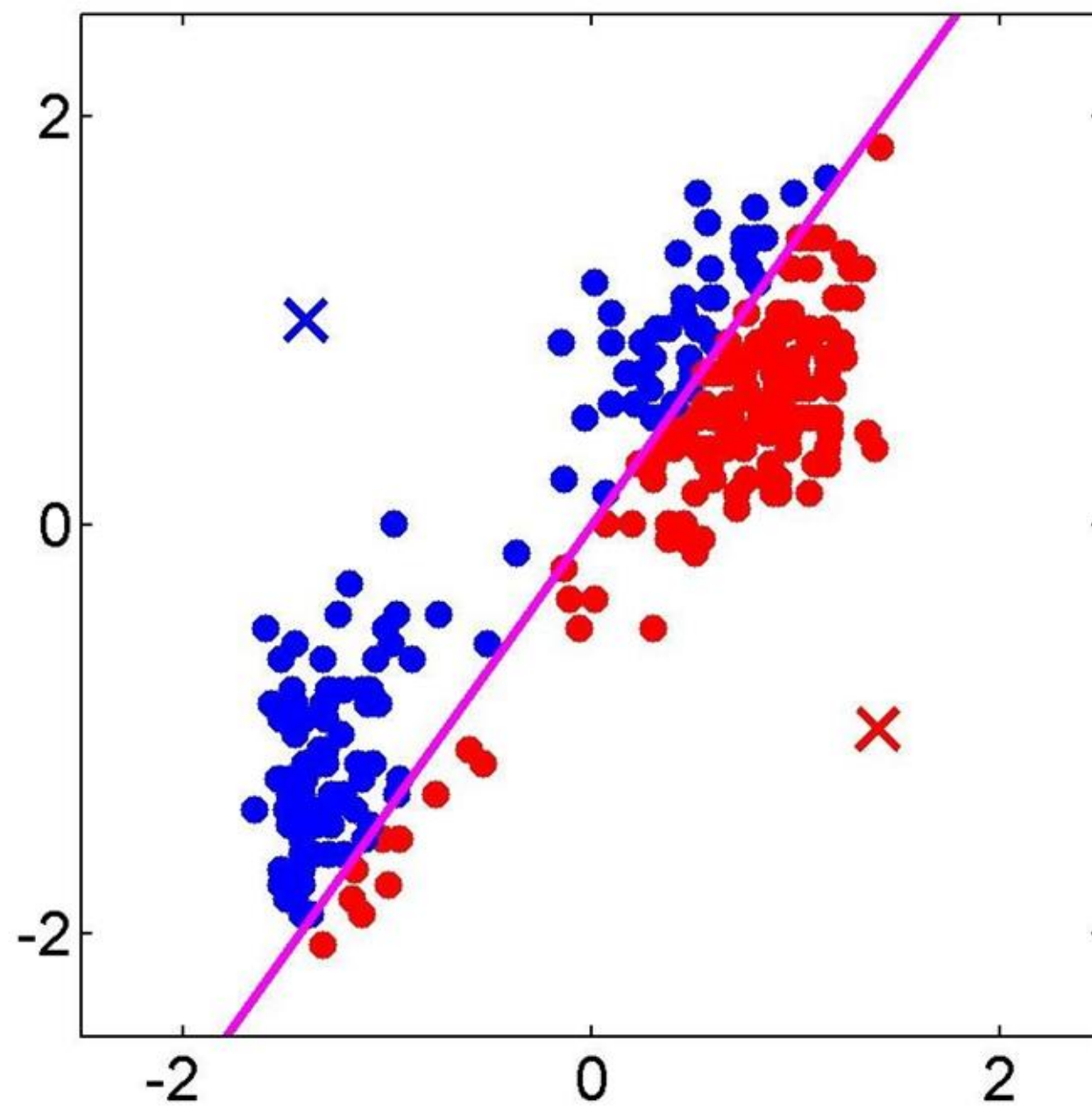
new centroid of C#1

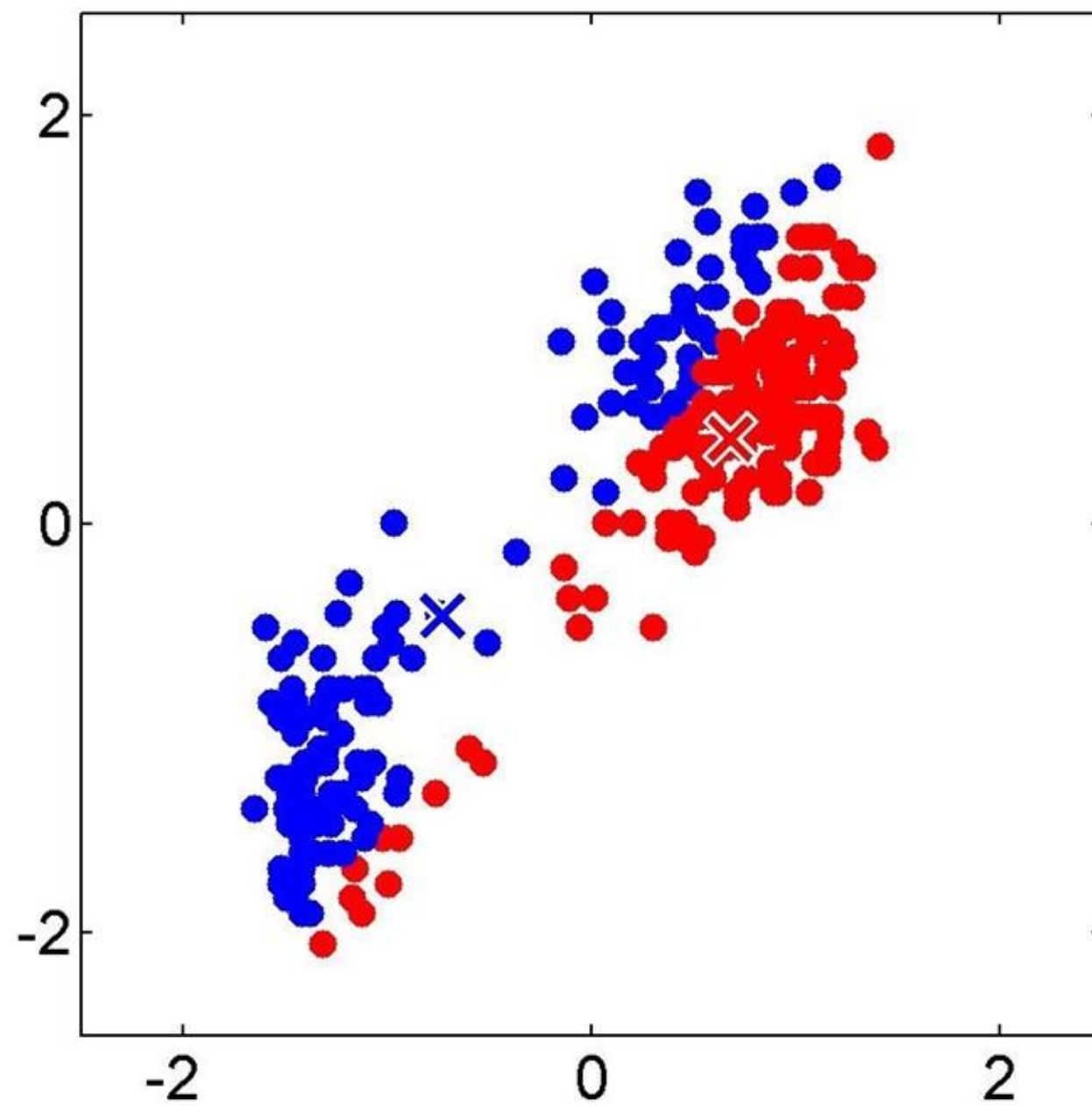
4	1.5	1.5
---	-----	-----

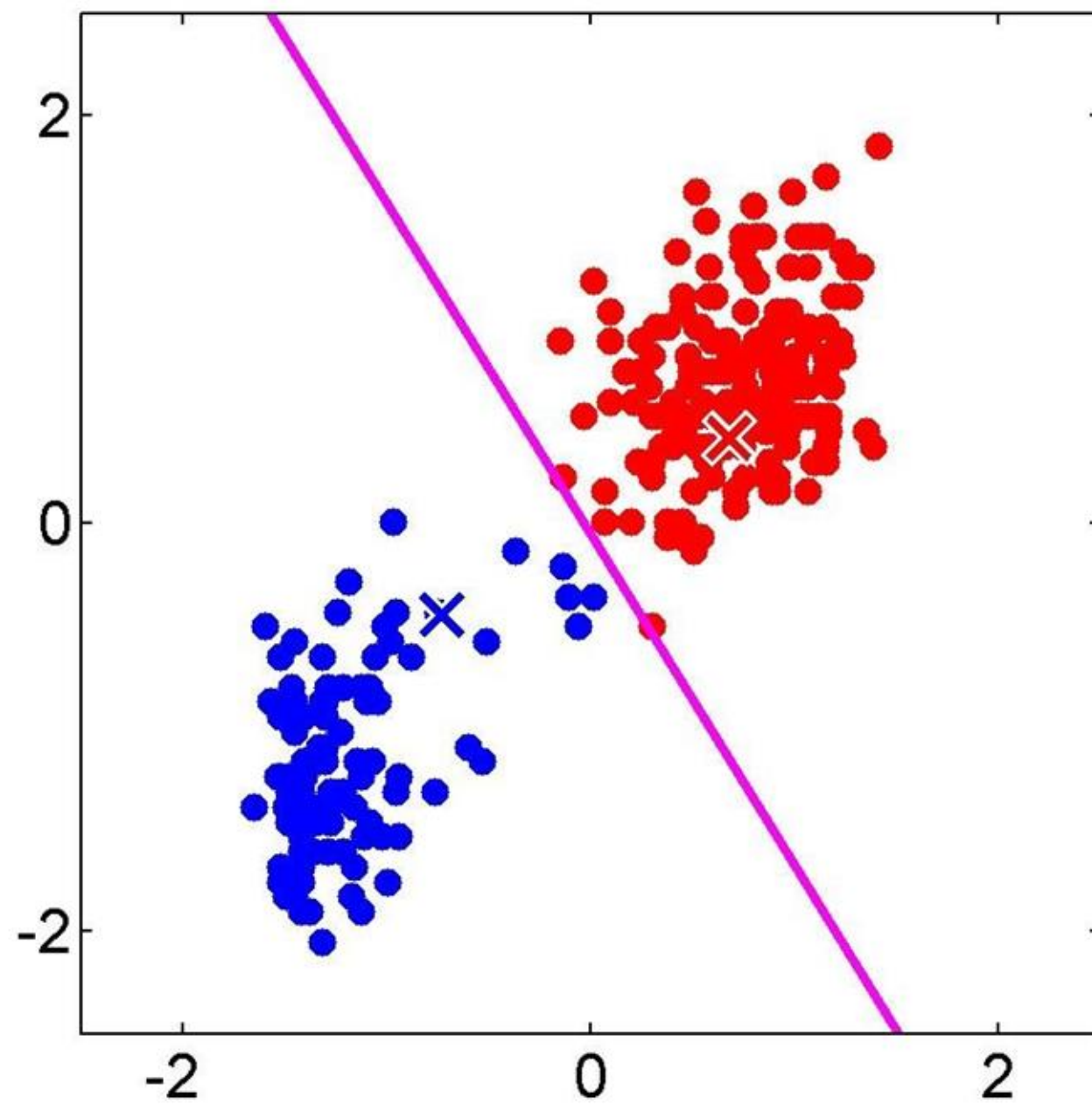
K-MEANS ALGORITHM – CONVERGENCE CRITERIA

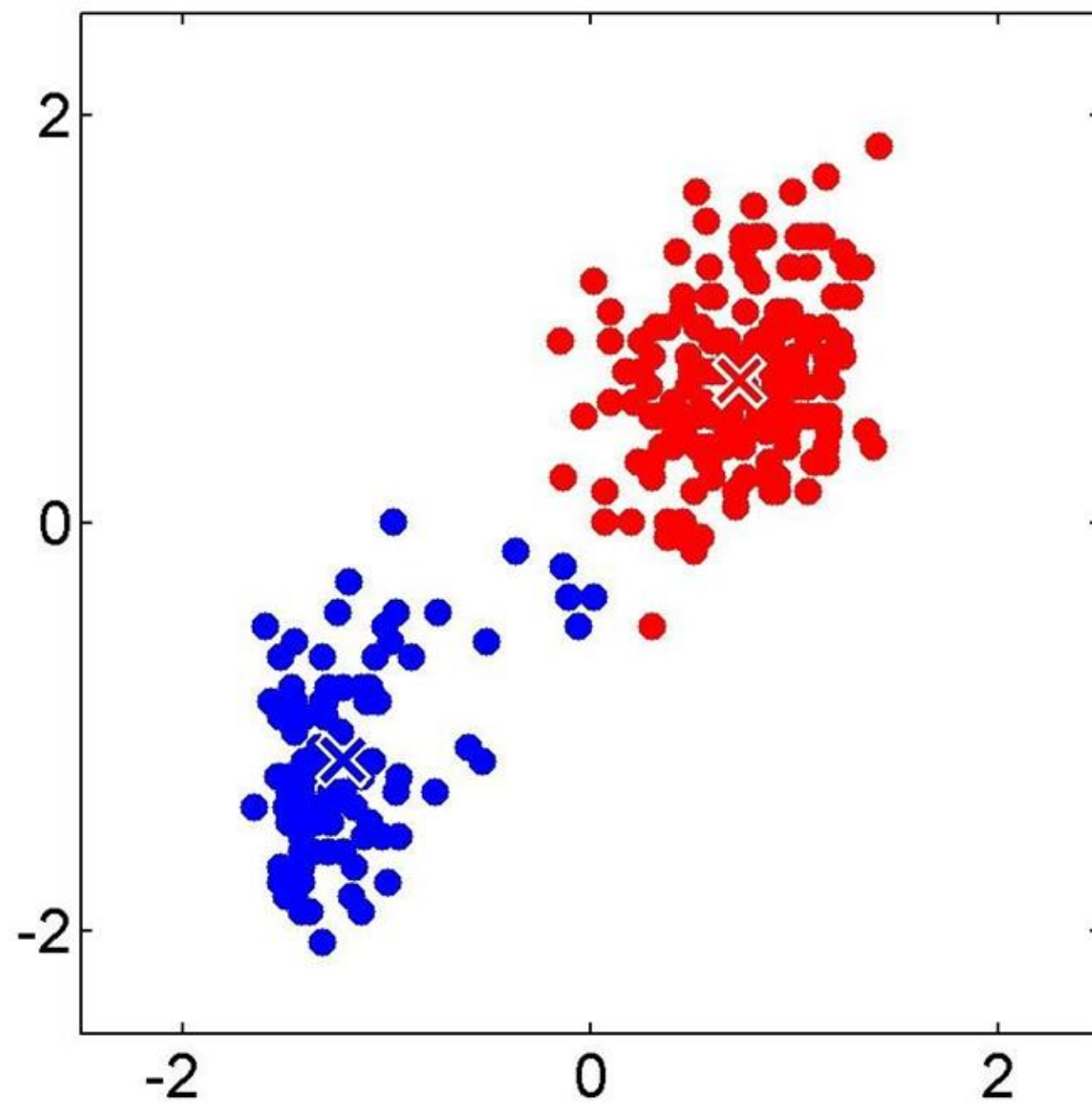
- The convergence criteria is the final step in the K-Means clustering algorithm, and it determines when the algorithm should stop iterating and the clusters are considered stable.
- The common convergence criteria –
 - the algorithm will stop after a predetermined number of iterations, regardless of whether the clusters have stabilized or not.
 - minimum change in centroids, where the algorithm will stop iterating when the centroids of all clusters change by less than a certain threshold value between consecutive iterations.
 - minimum change in the sum of squared errors (SSE) or the silhouette score.

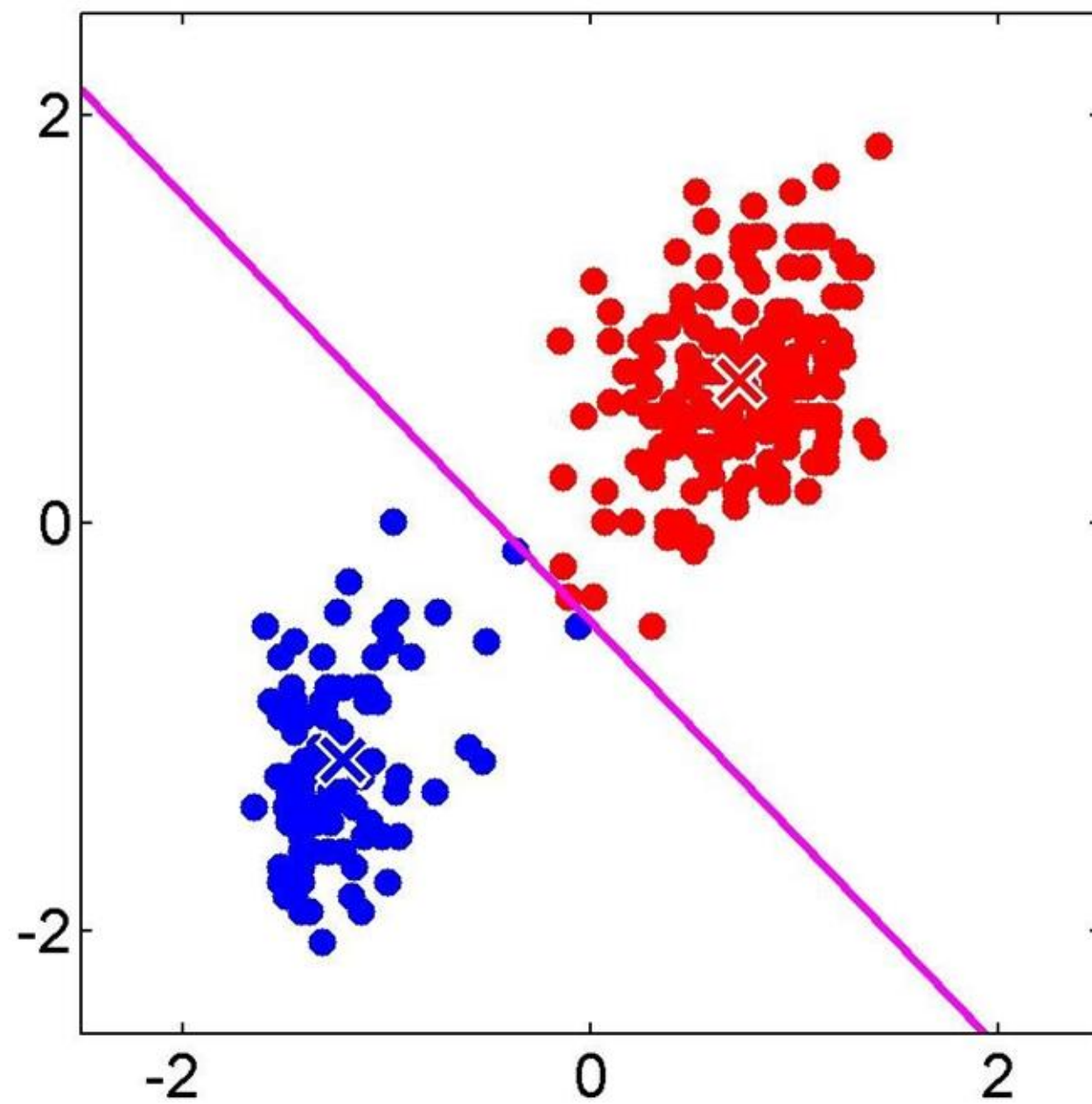


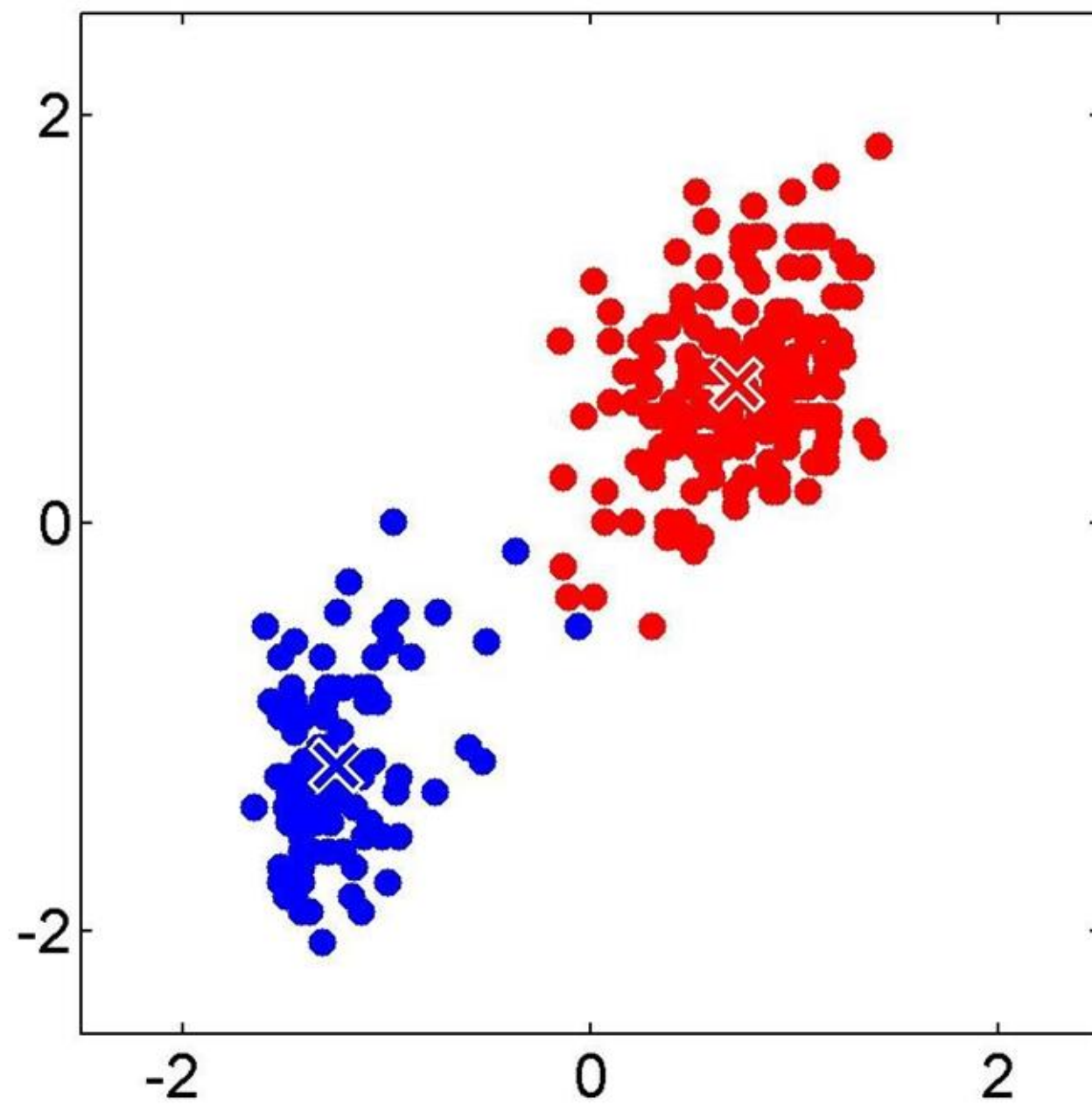


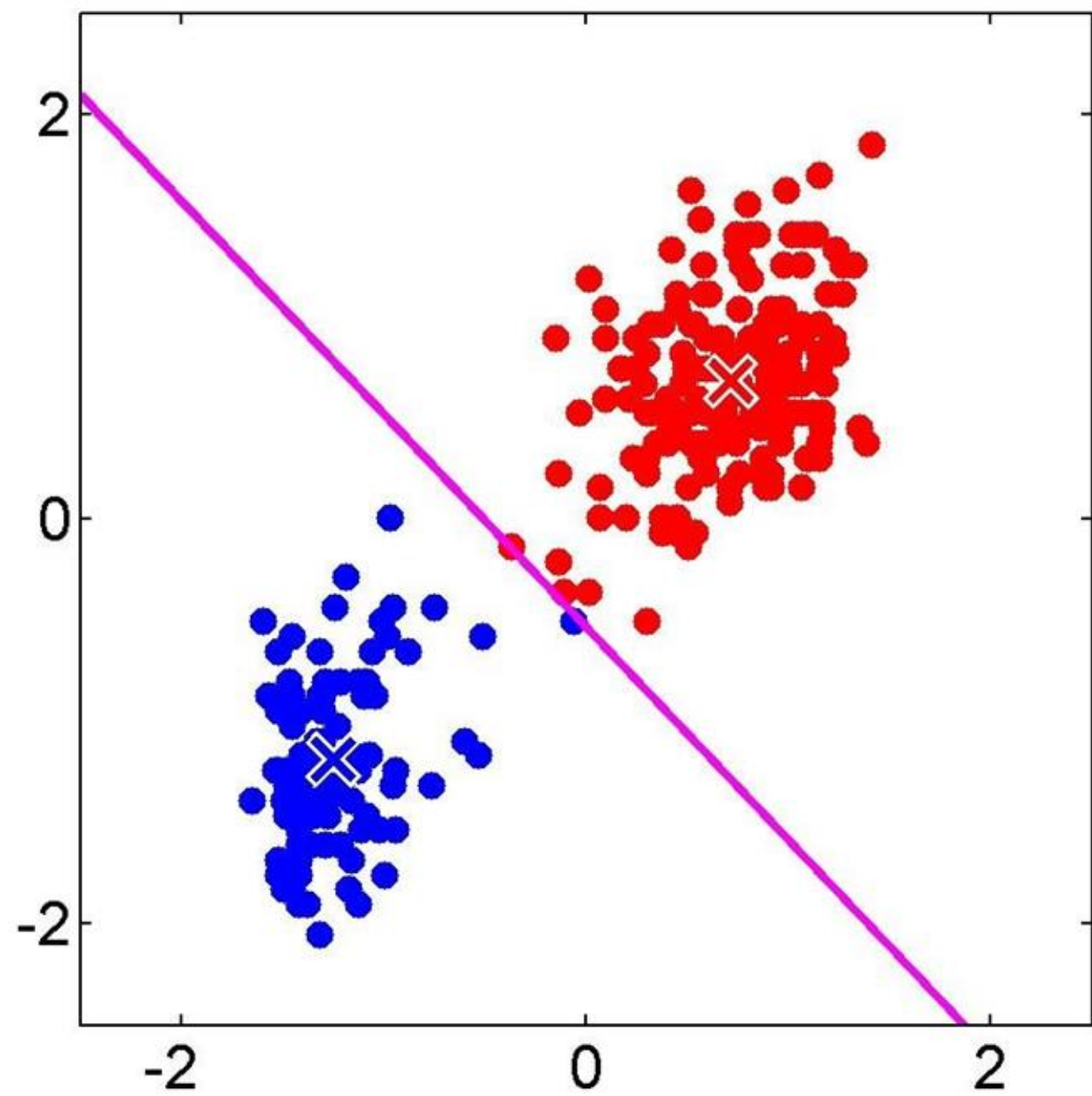


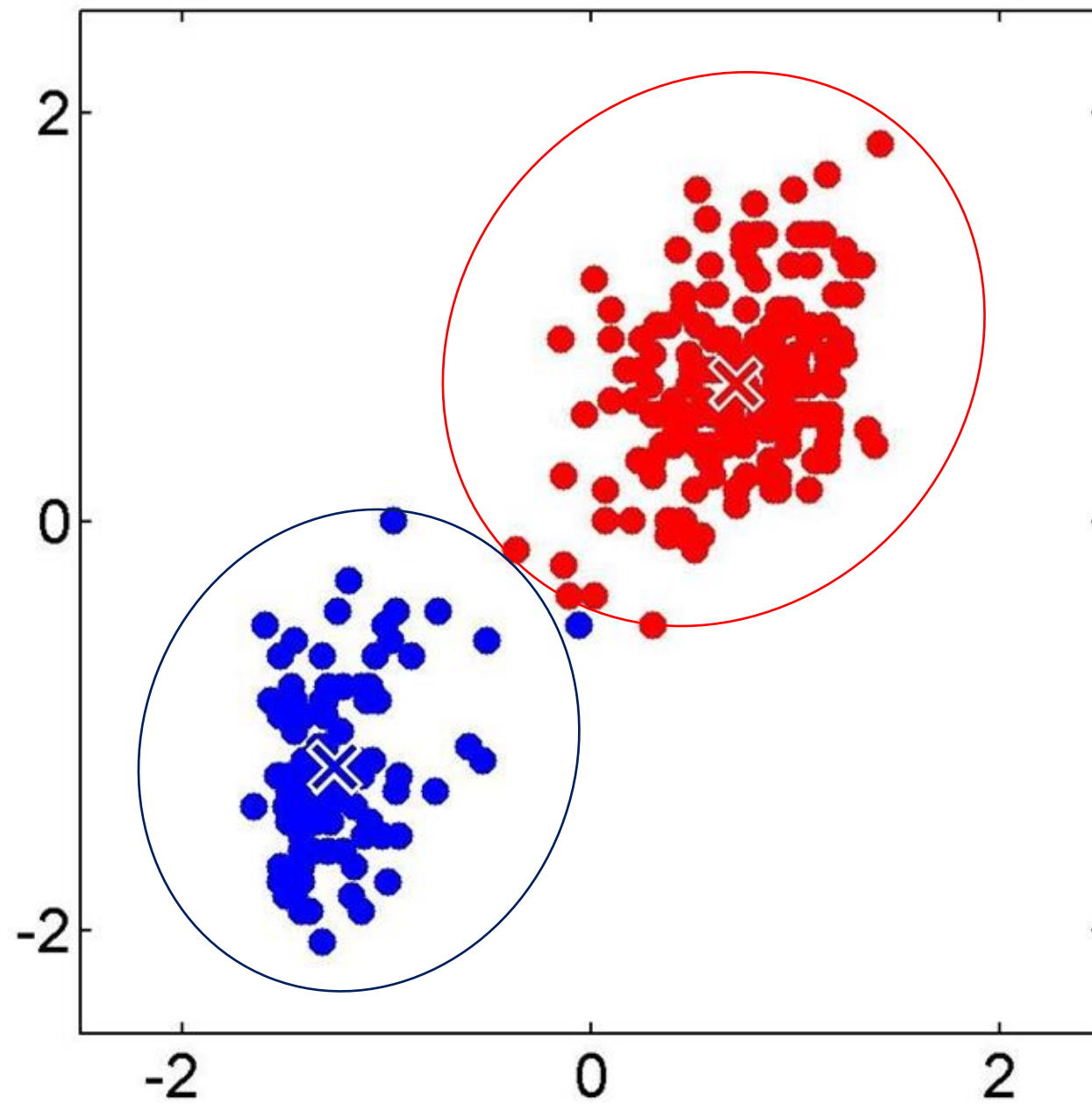






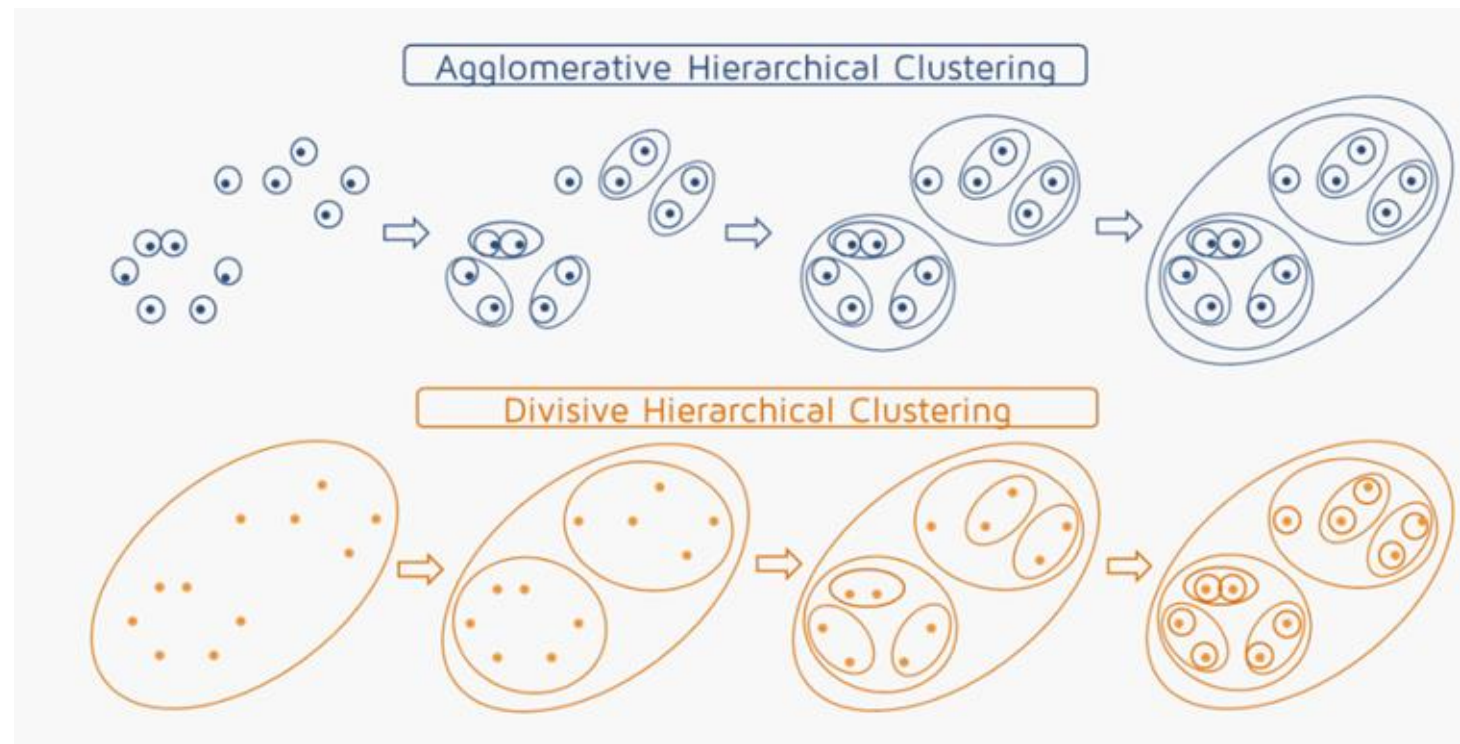






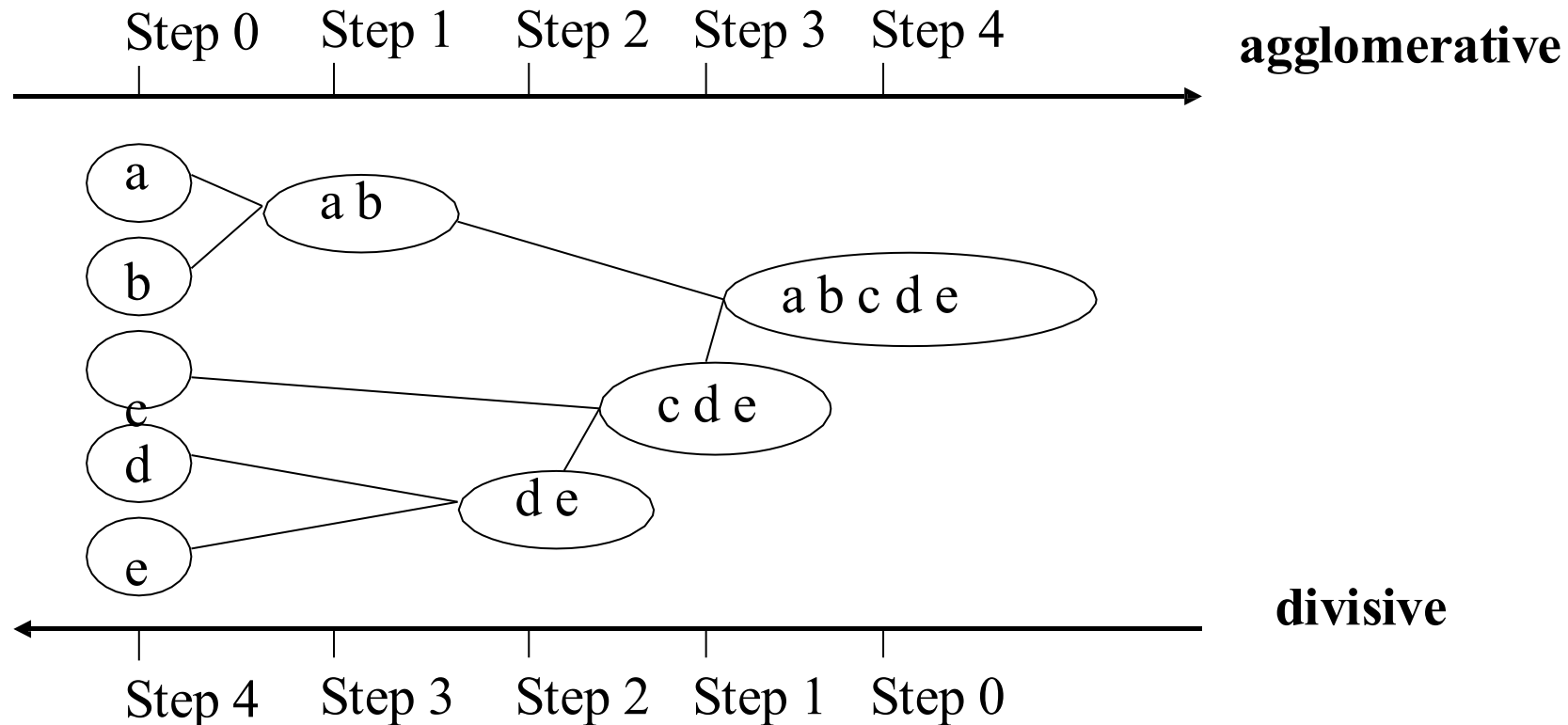
CLUSTERING ALGO – HIERARCHICAL CLUSTERING

- Type of clustering algorithm that creates a tree-like structure of clusters by recursively dividing or merging clusters based on the similarity between data points.
- Two main types:
 - Agglomerative
 - Divisive



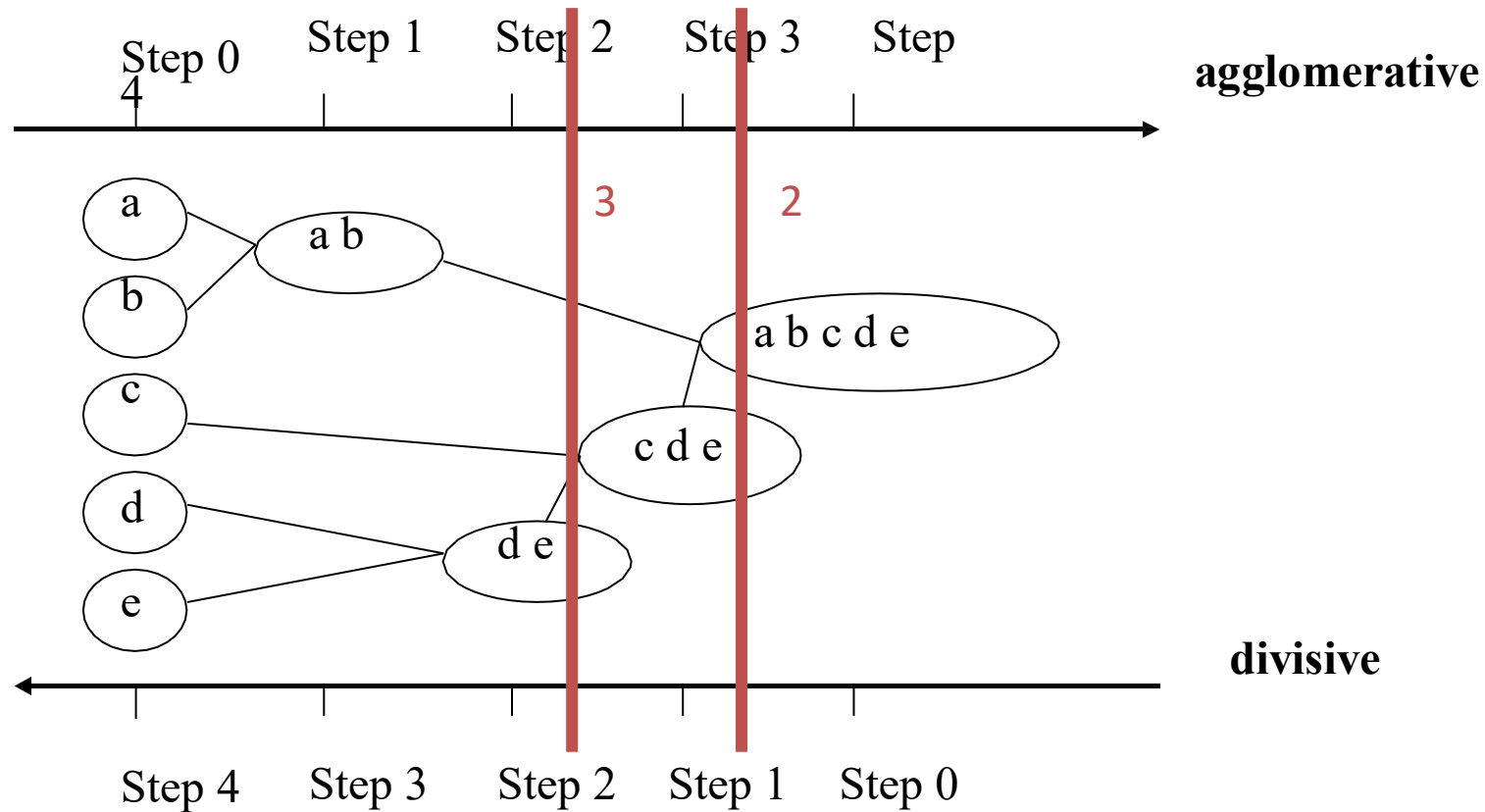
HIERARCHICAL CLUSTERING

- Two kinds of strategy
 - Bottom-up (agglomerative): Recursively merge two groups with the
 - smallest between-cluster dissimilarity (defined later on).
 - Top-down (divisive): In each step, split a least coherent cluster (e.g. largest diameter); splitting a cluster is also a clustering problem (usually done in a greedy way); less popular than bottom-up way.



HIERARCHICAL CLUSTERING

- User can choose a cut through the hierarchy to represent the most natural division into clusters
 - –e.g, Choose the cut where **intergroup** dissimilarity exceeds some threshold



HIERARCHICAL CLUSTERING

- Have to measure the dissimilarity for two disjoint groups G and H computed from pairwise dissimilarities $D(i, j)$ with $i \in G, j \in H$

- Single Linkage: tends to yield extended clusters.

$$D_{SL}(G, H) = \min_{i \in G, j \in H} D(i, j)$$

- Complete Linkage: tends to yield round clusters.

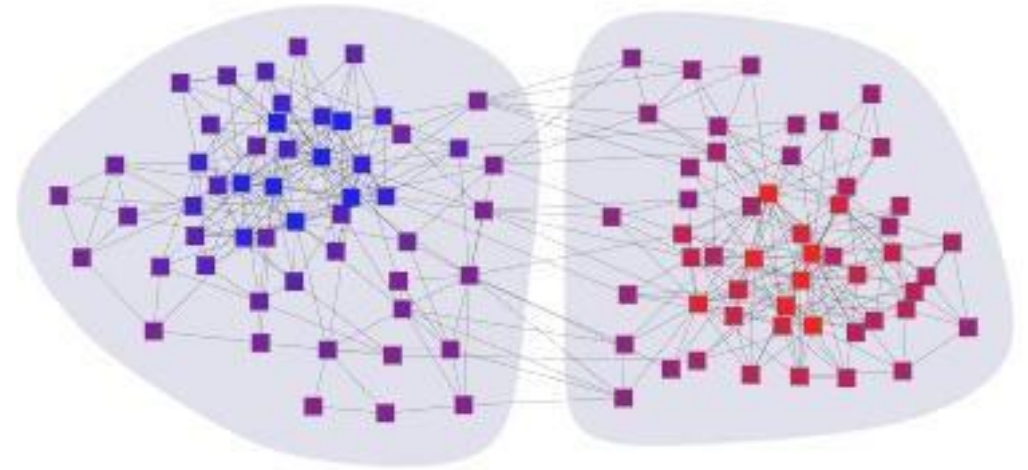
$$D_{CL}(G, H) = \max_{i \in G, j \in H} D(i, j)$$

- Group Average: tradeoff between them. Not invariant under monotone increasing transform.

$$D_{GA}(G, H) = \frac{1}{n_G n_H} \sum_{i \in G, j \in H} D(i, j)$$

CLUSTERING ALGO: SPECTRAL CLUSTERING

- Represent datapoints as the vertices V of a graph G .
- All pairs of vertices are connected by an edge E .
- Edges have weights W .
 - – Large weights mean that the adjacent vertices are very similar; small weights imply dissimilarity.



GRAPH PARTITIONING

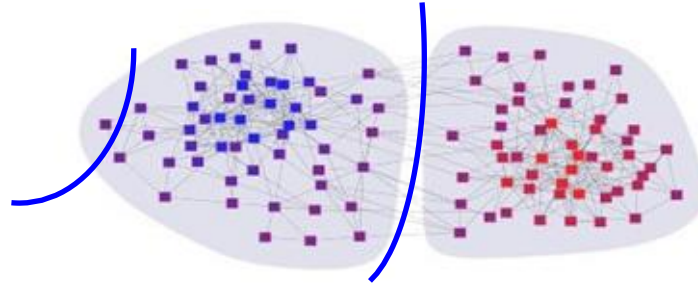
- Clustering on a graph is equivalent to partitioning the vertices of the graph.
- A loss function for a partition of V into sets A and B

$$cut(A, B) = \sum_{u \in A, v \in B} W_{u,v}$$

- In a good partition, vertices in different partitions will be dissimilar.
 - Mincut criterion: Find partition (A, B) that minimizes $cut(A, B)$

GRAPH PARTITIONING

- Mincut criterion ignores the size of the subgraphs formed.



- Normalized cut criterion favors balanced partitions.

$$Ncut(A, B) = \frac{cut(A, B)}{\sum_{u \in A, v \in V} W_{u,v}} + \frac{cut(A, B)}{\sum_{u \in B, v \in V} W_{u,v}}$$

- Minimizing the normalized cut criterion exactly is NP-hard

SPECTRAL CLUSTERING

- In spectral clustering, the data points are treated as nodes of a graph.
- Spectral clustering involves 3 steps:
 1. Compute a similarity graph
 2. Project the data onto a low-dimensional space
 3. Create clusters

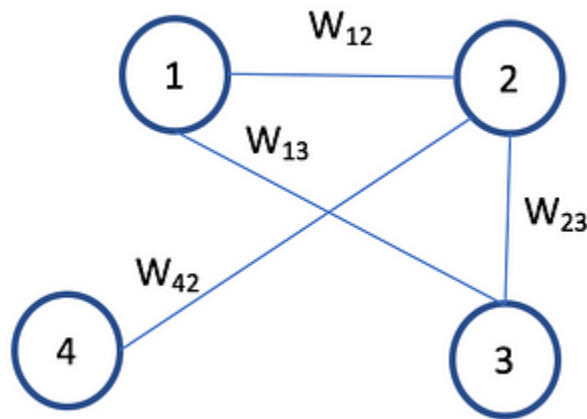
SPECTRAL CLUSTERING – STEP 1

- We first create an undirected graph $G = (V, E)$ with vertex set $V = \{v_1, v_2, \dots, v_n\} = 1, 2, \dots, n$ observations in the data.
- This can be represented by an adjacency matrix which has the similarity between each vertex as its elements.
- To perform that, we can opt to do any of these options:
 - **The ϵ -neighborhood graph**
 - **KNN graph**
 - **Fully connected graph**

$$s(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

SPECTRAL CLUSTERING – STEP 1

- Create an adjacency matrix for any of these graphs, $A_{ij} \sim 1$ when the points are close and $A_{ij} \rightarrow 0$ if the points are far apart.
- Consider the following graph with nodes 1 to 4, weights (or similarity) w_{ij} and its adjacency matrix:



	1	2	3	4
1	0	w_{12}	w_{13}	0
2	w_{12}	0	w_{23}	w_{24}
3	w_{13}	w_{23}	0	0
4	0	w_{24}	0	0

SPECTRAL CLUSTERING – STEP 2

- **Step 2 — Project the data onto a low-dimensional space:**
- Our goal then is to transform the space so that when the 2 points are close, they are always in same cluster, and when they are far apart, they are in different clusters.
- Compute the Graph Laplacian, which is just another matrix representation of a graph and can be useful in finding interesting properties of a graph.

SPECTRAL CLUSTERING – STEP 2

$$L = D - A,$$

where A is the adjacency matrix and D is the degree matrix such that

$$d_i = \sum_{\{j | (i,j) \in E\}} w_{ij} \quad \text{Thus, } L_{ij} = \begin{cases} d_i & \text{if } i = j \\ -w_{ij} & \text{if } (i,j) \in E \\ 0 & \text{if } (i,j) \notin E \end{cases}$$

Computing Graph Laplacian

$$\begin{aligned} d_1 &= w_{12} + w_{13} \\ d_2 &= w_{12} + w_{23} + w_{24} \\ d_3 &= w_{13} + w_{23} \\ d_4 &= w_{24} \end{aligned}$$

→ $L =$

	1	2	3	4
1	d_1	$-w_{12}$	$-w_{13}$	0
2	$-w_{12}$	d_2	$-w_{23}$	$-w_{24}$
3	$-w_{13}$	$-w_{23}$	d_3	0
4	0	$-w_{24}$	0	d_4

Graph Laplacian for our example above

SPECTRAL CLUSTERING – STEP 2

$$L\lambda = \lambda v$$

where v is the eigenvector of L corresponding to eigenvalue λ .

Thus we get eigenvalues $\{\lambda_1, \lambda_2, \dots, \lambda_n\}$ where $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$ and eigenvectors $\{v_1, v_2, \dots, v_n\}$.

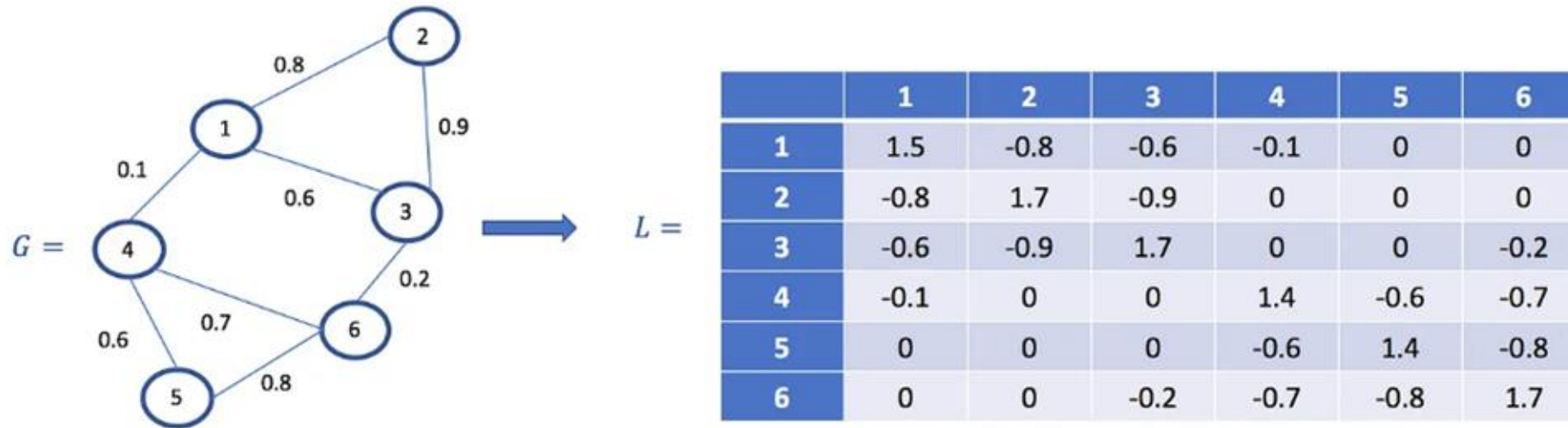
1. If L has eigenvalue 0 with k different eigenvectors, such that $0 = \lambda_1 = \lambda_2 = \dots = \lambda_k$, graph G has k connected components.
2. If G is connected, i.e. $0 = \lambda_1$ and $\lambda_2 > 0$, λ_2 is the algebraic connectivity of G . Thus, greater λ_2 , greater the connectivity.

Spectrum of the Laplacian

$$0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$$

SPECTRAL CLUSTERING – STEP 2

Let us consider an example with



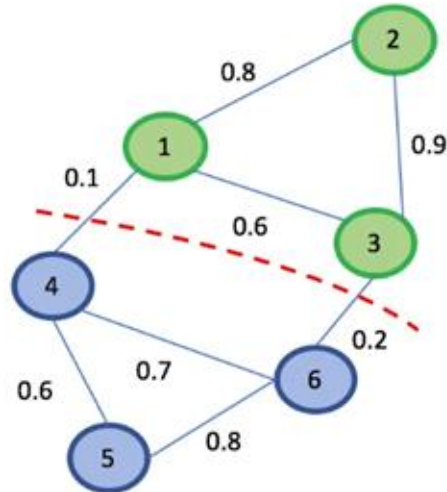
We then compute eigenvalues and eigenvectors for L

SPECTRAL CLUSTERING – STEP 3

- **Step 3 — Create clusters**
- For this step, we use the eigenvector corresponding to the 2nd eigenvalue to assign values to each node.
- On calculating, the 2nd eigenvalue is 0.189 and the corresponding eigenvector $v_2 = [0.41, 0.44, 0.37, -0.4, -0.45, -0.37]$.
- To get bipartite clustering (2 distinct clusters), we first assign each element of v_2 to the nodes such that $\{node1:0.41, node2:0.44, \dots node6: -0.37\}$.
- Then split the nodes such that all nodes with value > 0 are in one cluster, and all other nodes are in the other cluster.
- Thus, in this case, we get nodes 1, 2 & 3 in one cluster, and 4, 5 & 6 in the 2nd cluster.

SPECTRAL CLUSTERING – STEP 3

	v2
1	0.41
2	0.44
3	0.37
4	-0.40
5	-0.45
6	-0.37



Eigenvector v2 gives us bipartite clustering.

For k clusters, we have to modify our Laplacian to normalize it.

Thus we get:

$$L_{\text{norm}} = D^{-1/2} L D^{-1/2} =$$

	1	2	3	4	5	6
1	1.0	-0.5	-0.4	-0.1	0	0
2	-0.5	1.0	-0.5	0	0	0
3	-0.4	-0.5	1.0	0	0	-0.1
4	-0.1	0	0	1.0	-0.4	-0.5
5	0	0	0	-0.4	1.0	-0.5
6	0	0	-0.1	-0.5	-0.5	1.0

Normalized Laplacian — Ng, Jordan, Weiss

1. For k clusters, compute the first k eigenvectors $\{v_1, v_2, \dots, v_k\}$.
2. Stack the vectors vertically to form a matrix with the vectors as columns.
3. Represent every node by the corresponding row of this new matrix. These rows form the feature vectors of the nodes.
4. Use K-Means Clustering to now cluster these points into k clusters $\{C_1, C_2, \dots, C_k\}$.

PRINCIPLE COMPONENT ANALYSIS (PCA)

Home of the Bright, Land of the Brave
Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani

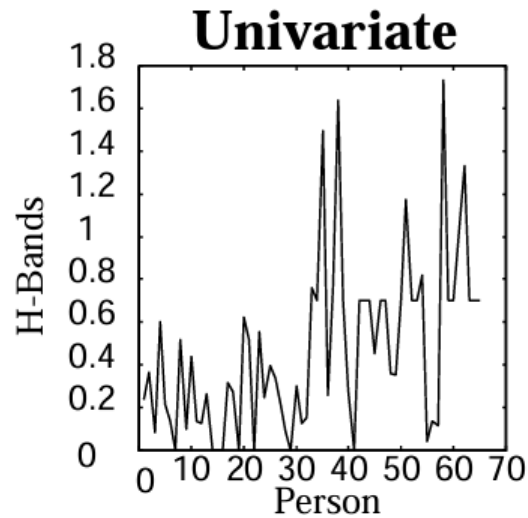


Problem: Hi-Dimensional Features

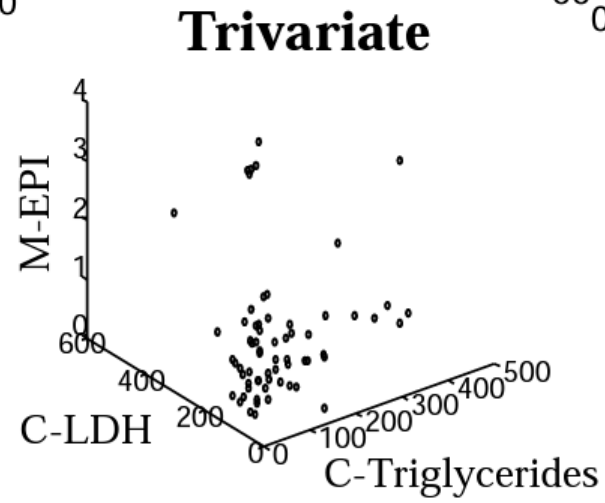
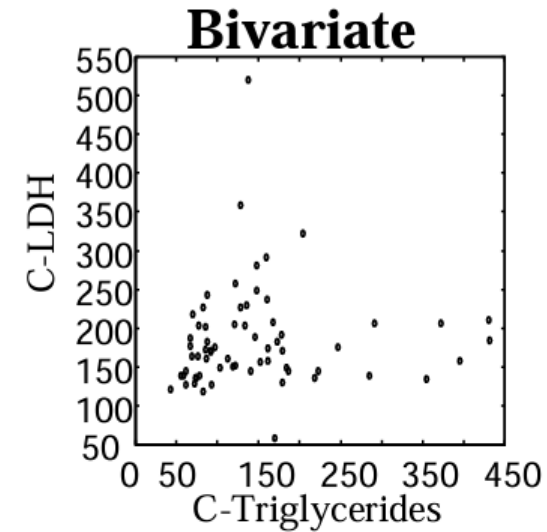
- Example:
 - Input data type: 53 blood and urine measurements (wet chemistry)
 - Samples: 65 people (33 alcoholics, 32 non-alcoholics)
- Data given in a matrix format

	H-WBC	H-RBC	H-Hgb	H-Hct	H-MCV	H-MCH	H-MCHC
A1	8.0000	4.8200	14.1000	41.0000	85.0000	29.0000	34.0000
A2	7.3000	5.0200	14.7000	43.0000	86.0000	29.0000	34.0000
A3	4.3000	4.4800	14.1000	41.0000	91.0000	32.0000	35.0000
A4	7.5000	4.4700	14.9000	45.0000	101.0000	33.0000	33.0000
A5	7.3000	5.5200	15.4000	46.0000	84.0000	28.0000	33.0000
A6	6.9000	4.8600	16.0000	47.0000	97.0000	33.0000	34.0000
A7	7.8000	4.6800	14.7000	43.0000	92.0000	31.0000	34.0000
A8	8.6000	4.8200	15.8000	42.0000	88.0000	33.0000	37.0000
A9	5.1000	4.7100	14.0000	43.0000	92.0000	30.0000	32.0000

Hi-Dimensional Data Representation



Multi-variate data distributions can be represented by distributions of pairs or triplets.



Hi-Dimensional Data Representation

- Issues
 - Any other better presentation?
 - Do we need all 53 dimensions?
 - How to find the BEST lower dimensional space that conveys maximum useful information?
- One answer: Find PRINCIPAL COMPONENTS
 - Goal: We wish to explain/summarize the underlying variance-covariance structure of a large set of variables through a few linear combinations of these variables.

Principal Component Analysis

- A method for finding the directions in high-dimensional data that contain information (dimension reduction)
- Applications :
 - Visualization
 - Exploration
 - Compression

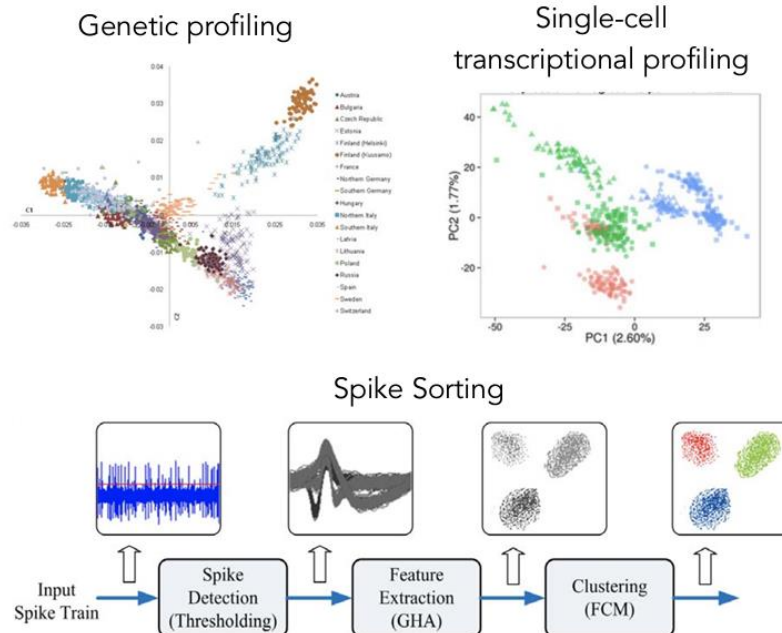


Figure 12 by Hwang, Wen-Jyi, et al., "Efficient Architecture for Spike Sorting in Reconfigurable Hardware," *Sensors* 13 no. 11 (2013): 14860-14887. MDPI Open Access. License: CC BY.

Eigenfaces



Screen shot @mikedusenberry.com. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>.

Eigenworm



Screen shot @JoVE. All rights reserved. This content is excluded from our Creative Commons license. For more information, see <https://ocw.mit.edu/help/faq-fair-use/>.

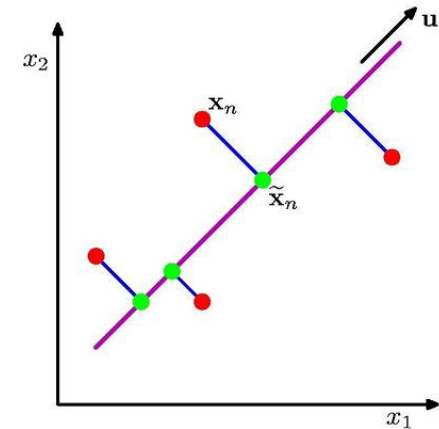
Principal Component Analysis

- Principal Component Analysis (PCA) is an unsupervised, non-parametric statistical technique primarily used for dimensionality reduction in machine learning.
- Also called the Karhunen-Loeve transform.
- PCA – data samples
 - Compute the mean, $x_1, \dots, x_N$
 - Computer the covariance: $\mu = (1/N) \sum_{i=1}^N x_i$.

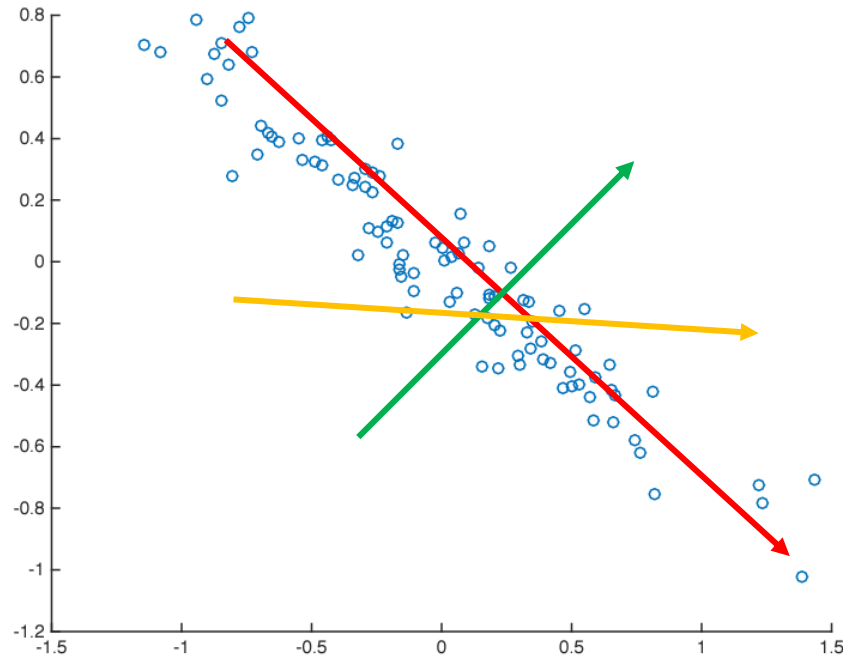
$$K = (1/N) \sum_{i=1}^N (x_i - \mu)(x_i - \mu)^T.$$

Principal Component Analysis

- Given N points in a 2-dimensional space, for large N ,
 - how does one project on to a 1-dimensional space?
 - broad trends in the data should be preserved and visualized
- Choose a line that fits the data so that the points be spread out well along the line, such that
 - The projection **maximizes variance** of projected data (purple line)
 - And minimizes mean squared distance between
 - data point and
 - projections (sum of blue lines)

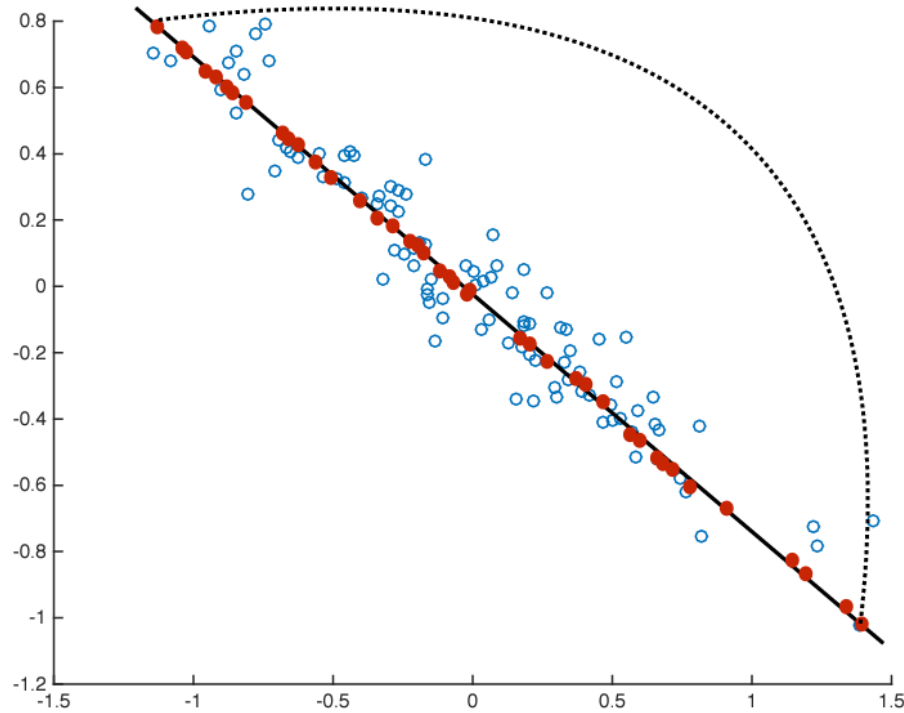


Maximum Variance Formulation



Which direction gives you the maximum variance?
Green / Orange / Red?

Maximum Variance Formulation

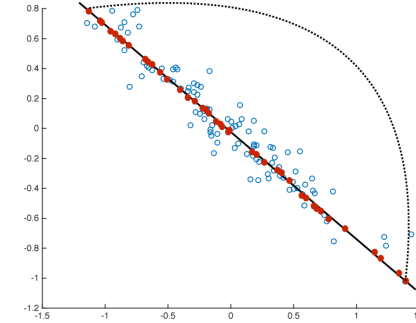


How do we know is this direction? How can we use math to compute?

Maximum Variance Formulation

$$\mathbf{X}_{m \times d} \xrightarrow{PCA} \mathbf{Z}_{m \times k} \quad (k < d)$$

$$\mathbf{Z}_{m \times k} = \mathbf{X}_{m \times d} \mathbf{U}_{d \times k}$$



$$\mathbf{Z} = [\mathbf{z}_1 \ \mathbf{z}_2 \ \cdots \ \mathbf{z}_k] \quad \mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_d] \quad \mathbf{U} = [\mathbf{u}_1 \ \mathbf{u}_2 \ \cdots \ \mathbf{u}_k]$$

$$\max_{\mathbf{u}_1} \text{Var}[\mathbf{z}_1] = \max_{\mathbf{u}_1} \text{Var}[\mathbf{X}\mathbf{u}_1]$$

$$\max_{\mathbf{u}_1} \mathbf{z}_1^T \mathbf{z}_1 = \max_{\mathbf{u}_1} \mathbf{u}_1^T \mathbf{X}^T \mathbf{X} \mathbf{u}_1$$

$$= \max_{\mathbf{u}_1} \mathbf{u}_1^T \Sigma_{\mathbf{X}} \mathbf{u}_1 \quad \Sigma_{\mathbf{X}} = \mathbf{X}^T \mathbf{X}: (\mathbf{N} \times \text{covariance of } \mathbf{X})$$

$$= \max_{\mathbf{u}_1} \mathbf{u}_1^T \Sigma_{\mathbf{X}} \mathbf{u}_1 \quad \text{s.t.} \quad \|\mathbf{u}_1\| = \mathbf{u}_1^T \mathbf{u}_1 = 1$$

Maximum Variance Formulation

Using Lagrange multipliers:

$$L(\mathbf{u}_1, \lambda_1) = \mathbf{u}_1^T \Sigma_{\mathbf{X}} \mathbf{u}_1 - \lambda_1 (\mathbf{u}_1^T \mathbf{u}_1 - 1)$$

$$\frac{\partial L}{\partial \mathbf{u}_1} = 2\Sigma_{\mathbf{X}} \mathbf{u}_1 - 2\lambda_1 \mathbf{u}_1 = 0$$

$$\Sigma_{\mathbf{X}} \mathbf{u}_1 = \lambda_1 \mathbf{u}_1 \quad \rightsquigarrow \quad \lambda_1 \text{ and } \mathbf{u}_1 \text{ are an eigenvalue-eigenvector pair of } \Sigma_{\mathbf{X}}$$

$$\text{Var}[\mathbf{z}_1] = \mathbf{u}_1^T \Sigma_{\mathbf{X}} \mathbf{u}_1 = \mathbf{u}_1^T \lambda_1 \mathbf{u}_1 = \lambda_1 \underbrace{\mathbf{u}_1^T \mathbf{u}_1}_1 = \lambda_1$$

For $\Sigma_{\mathbf{X}}$ there are d eigenvalue-eigenvector pairs:

$$\begin{array}{ccccccc} e_1 & > & e_2 & > & e_3 & > & \cdots & > & e_d \\ \mathbf{v}_1 & & \mathbf{v}_2 & & \mathbf{v}_3 & & \cdots & & \mathbf{v}_d \end{array}$$

Principal Component Analysis

$$\mathbf{X}_{m \times d} \xrightarrow{PCA} \mathbf{Z}_{m \times k} \quad (k < d)$$

$$\mathbf{Z}_{m \times k} = \mathbf{X}_{m \times d} \mathbf{U}_{d \times k}$$

$$\mathbf{Z} = [\mathbf{z}_1 \ \mathbf{z}_2 \ \cdots \ \mathbf{z}_k] \quad \mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \cdots \ \mathbf{x}_d] \quad \mathbf{U} = [\mathbf{u}_1 \ \mathbf{u}_2 \ \cdots \ \mathbf{u}_k]$$

The first principal direction $\mathbf{u}_1$ must be the eigenvector of $\Sigma_{\mathbf{X}}$ that corresponds to largest eigenvalue ($\mathbf{e}_1$).

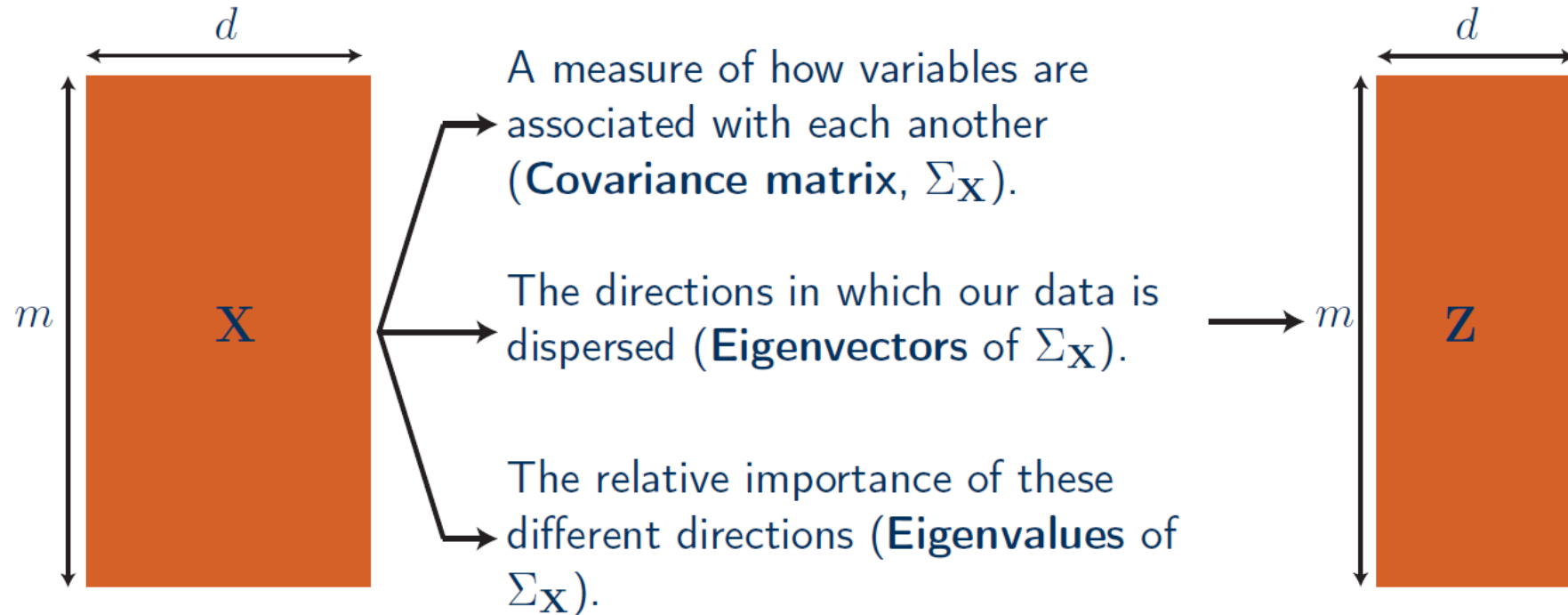
$$\mathbf{z}_1 = \mathbf{X}\mathbf{u}_1 \rightarrow \mathbf{z}_1 = \mathbf{X}\mathbf{v}_1$$

What about other principal components? $\mathbf{z}_{2 \dots k} = \mathbf{X}\mathbf{u}_{2 \dots k} \stackrel{?}{=} \mathbf{X}\mathbf{v}_{2 \dots k}$

Each new principal direction $\mathbf{u}_i$ should:

- maximise $\text{Var}[\mathbf{z}_i]$;
- be orthogonal to all other $\mathbf{u}_j$; extracting something new from $\mathbf{X}$.

Principal Component Analysis



PCA linearly combines our variables and allows us to drop projections that are less informative.

Principal Component Analysis

Advantages:

- Correlated features removed
- Speeds up other machine learning algorithms
- Improves visualization

Disadvantages:

- Data normalization required before performing the PCA
- We may lose some valuable information
- Major components may be difficult to understand

Principal Component Analysis

1. Doing PCA to avoid overfitting is a bad idea. Instead use regularisation.
2. Doing PCA to for dimensionality reduction before classification is also a bad idea.
Instead use a method called, linear discriminant analysis (LDA).

Application of PCA: Face Recognition

Task: Identify specific person, based on facial image regardless of wearing glasses, lighting,...

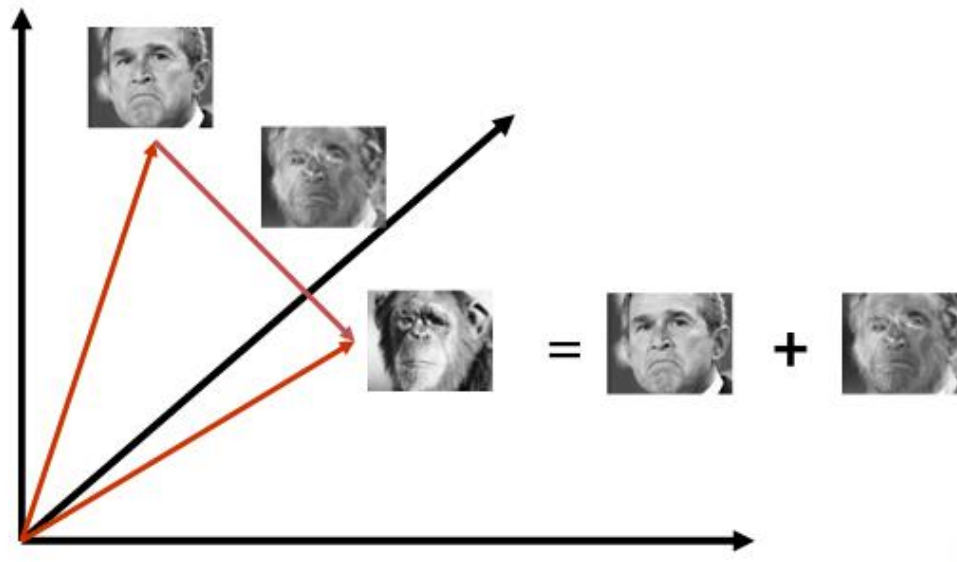
Can we use all the given 256 x 256 pixels?

→ $2^{16} = 65,536$ for 256 gray-level images



Application of PCA: Face Recognition

- An image is a point in a high dimensional space.
 - An $N \times M$ image is a point in R^{NM}
 - We can define vectors in this space as we did in the 2D case



Key Idea

- Images in the possible set $\mathcal{X} = \{\hat{x}_{RL}^P\}$ are highly correlated.
- So, compress them to a low-dimensional subspace that captures key appearance characteristics
- EIGENFACES: [Turk and Pentland] → USE PCA

EIGENFACES



Eigenfaces look somewhat like generic faces.

EIGENFACES

- Eigenfaces are the eigenvectors of the covariance matrix of the vector space of human faces
- Eigenfaces are the 'standardized face ingredients' derived from the statistical analysis of many pictures of human faces
- A human face may be considered to be a combination of these standardized faces

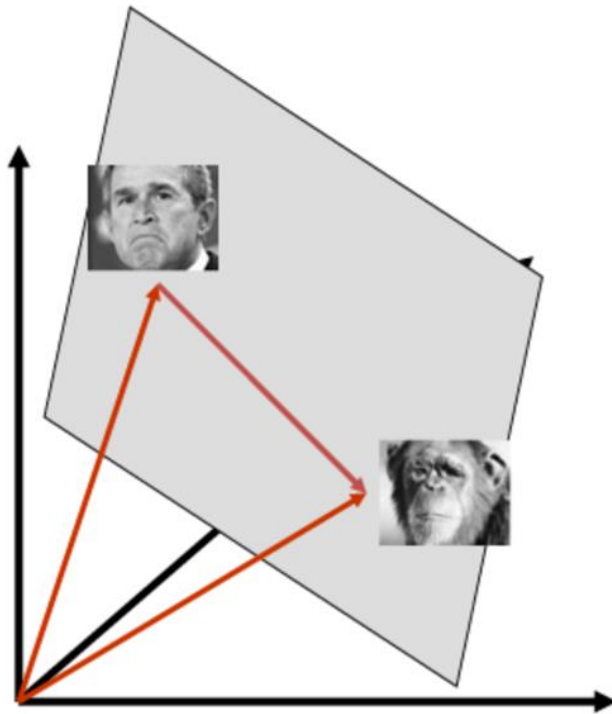
Generating EIGENFACES

1. Large set of images of human faces is taken.
2. The images are normalized to line up the eyes, mouths and other features.
3. The eigenvectors of the covariance matrix of the face image vectors are then extracted.
4. These eigenvectors are called eigenfaces

EIGENFACES for Face Recognition

- When properly weighted, eigenface scan be summed together to create an approximate gray-scale rendering of a human face.
- Remarkably few eigenvector terms are needed to give a fair likeness of most people's faces.
- Hence eigenfaces provide a means of applying data compression to faces for identification purposes.

Dimension Reduction



The set of faces is a “subspace” of the set of images

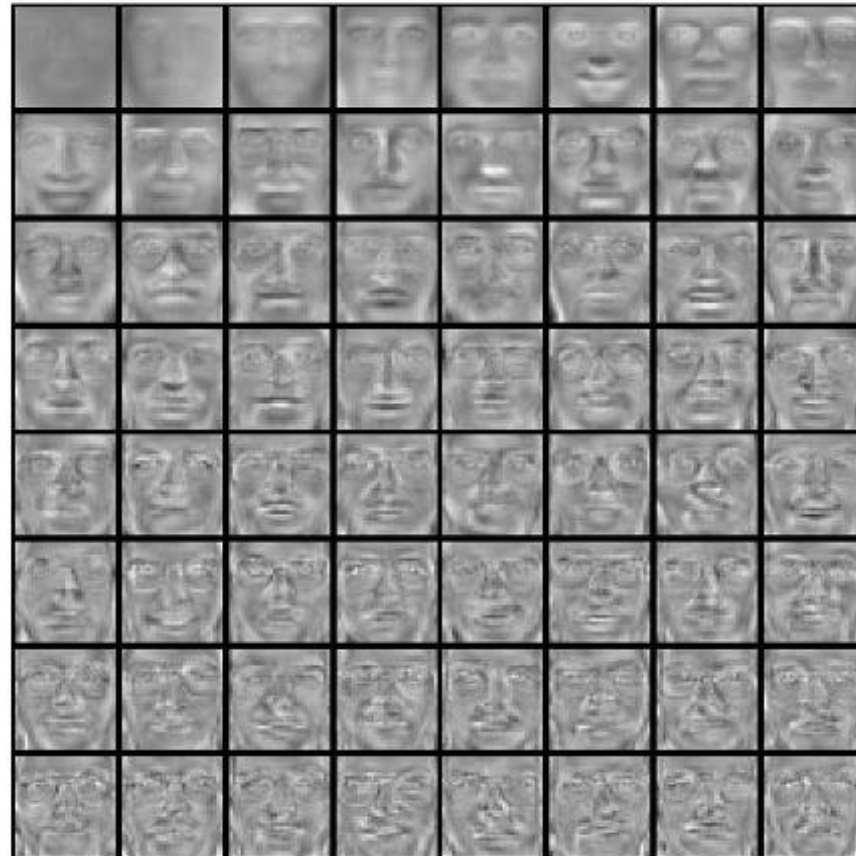
- Suppose it is K dimensional.
- We can find the best subspace using PCA
- This is like fitting a “hyper-plane” to the set of faces
 - spanned by vectors $v_1, v_2, \dots, v_K$

$$\text{Any face: } \mathbf{x} \approx \bar{\mathbf{x}} + a_1 \mathbf{v}_1 + a_2 \mathbf{v}_2 + \dots + a_k \mathbf{v}_k$$

Eigenfaces Example

Top eigenvectors: $\mathbf{e}_1, \dots, \mathbf{e}_k$

Mean: μ

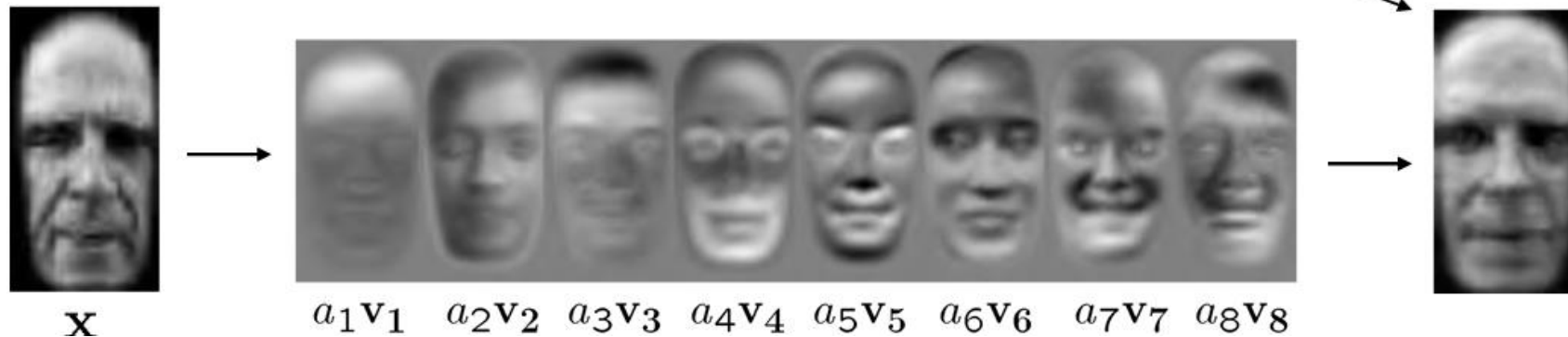


Projecting on to Eigenfaces

- The eigenfaces $v_1, \dots, v_K$ span the space of faces

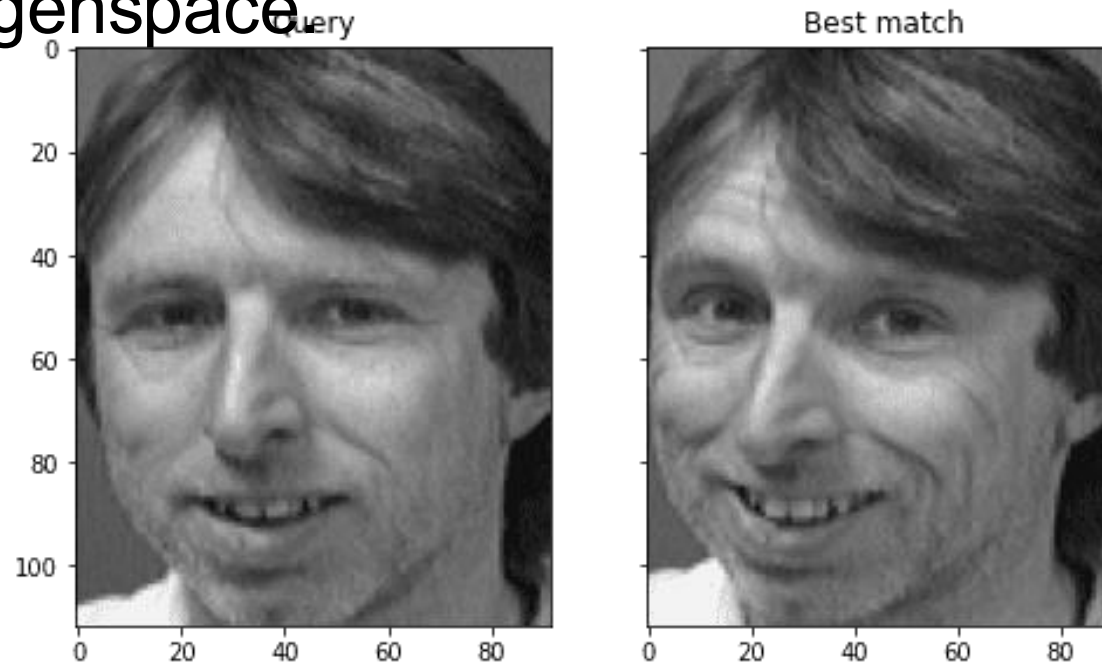
$$\mathbf{x} \rightarrow \left(\underbrace{(\mathbf{x} - \bar{\mathbf{x}}) \cdot \mathbf{v}_1}_{a_1}, \underbrace{(\mathbf{x} - \bar{\mathbf{x}}) \cdot \mathbf{v}_2}_{a_2}, \dots, \underbrace{(\mathbf{x} - \bar{\mathbf{x}}) \cdot \mathbf{v}_K}_{a_K} \right)$$

$$\mathbf{x} \approx \bar{\mathbf{x}} + a_1 \mathbf{v}_1 + a_2 \mathbf{v}_2 + \dots + a_K \mathbf{v}_K$$



Face Recognition

- Map the Query image to eigenspace.
- Find the closest match in the same class by computing the Euclidean distance in eigenspace.



THANK YOU

Home of the Bright, Land of the Brave
Di Sini Bermulanya Pintar, Tanah Tumpahnya Berani



www.um.edu.my



[universityofmalaya](https://www.facebook.com/universityofmalaya)



[unimalaya](https://www.instagram.com/unimalaya)



[uniofmalaya](https://www.youtube.com/uniofmalaya)



UNIVERSITI
MALAYA